

TACM — Tool Access Control Module

Parent Standard: Harness Architecture Standard

Parent Architecture: **Execution Layer — Enforcement Standard**

Category: Governance & Enforcement

Subcategory: Tool Access Control

Type: Execution Containment Module

Version: 1.0

Status: Canonical · Open Module

Effective Date: 9 May 2026

Compatibility: OOF® Methodology OS™ · **Harness Architecture Standard** ·

Execution Layer — Enforcement Standard · RIS · INTEGROS® · OGL™ · ART · OIB™

Authority: OOF®

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

Tool Access Control Module defines the structural conditions under which a system may access, invoke, restrict, sequence, or deny tools, interfaces, functions, or execution-capable resources within a **contained execution** environment.

A system satisfies TACM only if:

- tool access is explicitly defined
- available and unavailable tools are distinguishable
- invocation conditions remain bounded by governed rules
- unauthorized or out-of-scope tool use is restricted
- tool access does not silently expand through delegation, runtime change, or indirect execution paths

A system that can invoke tools without governed access conditions does not satisfy TACM.

Module Function

TACM defines the controlled tool-access layer of execution containment.

It ensures that tools are not treated as neutral capabilities available by default, but as governed execution surfaces that must remain bounded, permission-linked, and operationally restricted.

The module applies wherever autonomous systems may call:

- tools
- APIs
- execution functions
- filesystems
- code runtimes

- external services
 - command environments
 - action-triggering interfaces
-

Minimum Implementation Framework (MIF)

Step 1 — Define the Tool Surface

The organization must define which tools or execution-capable interfaces exist within the contained environment.

Minimum requirement:

- tool classes are explicit
 - executable and non-executable interfaces are distinguishable
 - undefined tool surfaces are excluded from valid operation
-

Step 2 — Define Access Eligibility

The system must define who or what may access each tool.

Minimum requirement:

- tool eligibility is explicit
 - valid callers are identifiable
 - unauthorized entities are excluded from valid tool invocation
-

Step 3 — Define Invocation Conditions

The system must define under what conditions a tool may be invoked.

Minimum requirement:

- invocation triggers are explicit
 - tool use is not open-ended by default
 - context-free or unrestricted invocation is excluded from valid access logic
-

Step 4 — Define Restriction and Denial Logic

The system must define how tool access is limited, denied, or escalated.

Minimum requirement:

- restricted and prohibited tool conditions are explicit
 - denial logic is enforceable
 - silent expansion of access is excluded from valid containment
-

Step 5 — Preserve Tool Access Traceability

The system must preserve visibility of who invoked what, when, under which condition, and through which path.

Minimum requirement:

- tool invocation path is traceable
- later review can reconstruct access history
- indirect or chained tool access remains visible

Step 6 — Restrict Invalid Tool Use

The system must not be treated as valid if tools are invoked outside governed scope, through hidden access paths, or without explicit bounded conditions.

Minimum requirement:

- invalid tool access conditions are identifiable
- unauthorized or out-of-scope invocation is blocked or invalidated
- execution convenience does not override tool-access governance

Use Case 1 — AI Agent with Mixed Tool Surface

Scenario

An AI agent operates inside a contained environment with access to selected tools, but must not invoke external functions or higher-impact tools outside approved scope.

Application

TACM is used to ensure:

- available tools are explicitly defined
- only valid callers may invoke them
- access conditions remain bounded
- unauthorized tool expansion is blocked

Result

The organization gains a stronger basis for preventing **authorized execution** from turning into uncontrolled tool use.

Use Case 2 — Enterprise Automation with Sensitive Interfaces

Scenario

A system may interact with documents, internal APIs, workflow triggers, or external services, but only under specific operational and containment conditions.

Application

TACM is used to ensure:

- tool surface is governed
 - invocation conditions are explicit
 - denial and restriction logic remain enforceable
 - access history is traceable across the contained environment
-

Result

The organization gains a structurally governed tool-access layer instead of relying on assumed good behavior inside an execution environment.

Canonical Closing Statement

If tool access is not governed, execution containment does not exist.

Related Documents

→ Harness Architecture Standard

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>